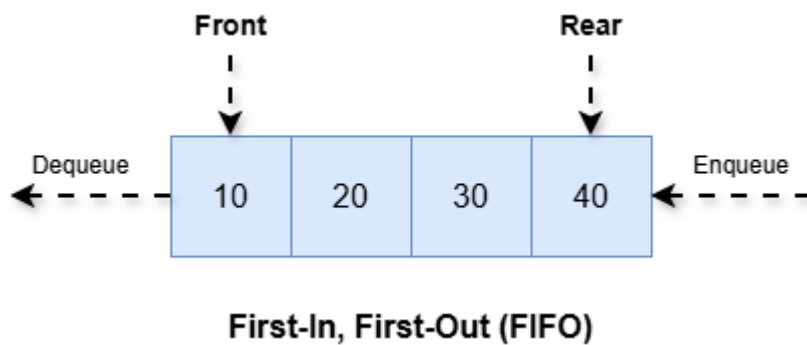


Queue Data Structure

Queue Data Structure

Queue

1. Linear data structure
2. Works on **FIFO (First In First Out)** principle



Operations are:

1. Enqueue (Insert Element)

Add element to the **rear (end)** of queue.

```
queue = []  
  
queue.append(10)  
queue.append(20)  
queue.append(30)  
  
print(queue)  
# O/P:- [10, 20, 30]
```

2. Dequeue (Remove Element)

Remove element from the **front** of queue.

```
queue.pop(0)  
  
print(queue)  
# O/P:- [20, 30]
```

3. isEmpty()

Check queue empty or not.

```
if len(queue) == 0:
    print("Queue is empty")
else:
    print("Queue is not empty")
```

4. isFull()

Check queue full or not.

```
if len(queue) == 10:
    print("Queue is full")
else:
    print("Queue is not full")
```

5. Front / Peek

See first element without removing.

```
print(queue[0])
```

O/P:- 20

Queue with Functions

```
def create_queue():
    return []

def enqueue(queue, item):
    queue.append(item)

def dequeue(queue):
    if len(queue) == 0:
        raise IndexError("Queue Underflow")
    return queue.pop(0)

def front(queue):
    if not queue:
        print("Queue is empty. Cannot peek.")
        return None
    return queue[0]

def isEmpty(queue):
    return len(queue) == 0

def isFull(queue, capacity):
    return len(queue) == capacity
```

```
# ---- Usage ----
queue = create_queue()

item = eval(input("Enter item to enqueue: "))
enqueue(queue, item)

removed_item = dequeue(queue)
print(f"Dequeued item: {removed_item}")

front_item = front(queue)
print(f"Front item: {front_item}")

print(f"Is queue empty? {isEmpty(queue)}")

capacity = eval(input("Enter capacity: "))
print(f"Is queue full? {isFull(queue, capacity)}")
```

Queue with Class

```
class Queue:
    def __init__(self, capacity=None):
        self.queue = []
        self.capacity = capacity

    def enqueue(self, item):
        if self.capacity is not None and len(self.queue) == self.capacity:
            print("Queue is full. Cannot enqueue.")
            return
        self.queue.append(item)

    def dequeue(self):
        if not self.queue:
            print("Queue Underflow. Cannot dequeue.")
            return None
        return self.queue.pop(0)

    def front(self):
        if not self.queue:
            print("Queue is empty.")
            return None
        return self.queue[0]
```

```
def isEmpty(self):
    return len(self.queue) == 0

def isFull(self):
    if self.capacity is None:
        return False
    return len(self.queue) == self.capacity

# ---- Usage ----
capacity = int(input("Enter capacity: "))
q = Queue(capacity)

item = eval(input("Enter item to enqueue: "))
q.enqueue(item)

removed_item = q.dequeue()
print(f"Dequeued item: {removed_item}")

front_item = q.front()
print(f"Front item: {front_item}")

print(f"Is queue empty? {q.isEmpty()}")
print(f"Is queue full? {q.isFull()}")
```